

Lecture 16: Advanced RDBMS Concepts

Views and Indexes

Amit Kumar Dhar

September 17, 2025

SQL Views

Views: Definition and Example

Definition

A **view** is a virtual table based on the result-set of an SQL statement. It contains rows and columns, just like a real table, but it does not store the data itself. The data is fetched from the base tables each time the view is queried.



Figure 1: A view is a virtual representation of data from one or more tables.

Views: Example SQL

Base Table: Students

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Department VARCHAR(50),  
    GPA DECIMAL(3, 2)  
);
```

Creating a View

This view shows only students from the 'CS' department.

```
CREATE VIEW CS_Students_View AS  
SELECT StudentID, Name, GPA  
FROM Students  
WHERE Department = 'CS';
```

Creating, Querying, and Deleting Views

Creating a View

The CREATE VIEW statement is used to create a new view.

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Querying a View

You can query a view just like you would query a regular table.

```
SELECT * FROM CS_Students_View WHERE GPA > 3.5;
```

Deleting a View

The DROP VIEW statement is used to delete a view.

```
DROP VIEW view_name;
```

Updatable Views

Updatable Views: Definition and Conditions

Definition

An **updatable view** is a view that allows DML operations (INSERT, UPDATE, DELETE). These operations are passed through to the underlying base table.

Conditions for a View to be Updatable

A view is generally updatable if it meets these conditions:

- The SELECT list does not contain DISTINCT.
- The query is based on a single table.
- The query does not use aggregate functions (e.g., MIN, MAX, SUM).
- The query does not contain GROUP BY or HAVING clauses.
- The query does not use subqueries in the SELECT list.

Updatable Views: Example

Creating an Updatable View

```
CREATE VIEW High_GPA_Students AS  
SELECT StudentID, Name, GPA  
FROM Students  
WHERE Department = 'EE';
```

Updating Through the View

This UPDATE statement will modify the underlying Students table.

```
UPDATE High_GPA_Students  
SET GPA = 3.9  
WHERE StudentID = 105; -- Assuming 105 is an EE student
```



Materialized Views

Materialized Views: Motivation and Definition

Motivation

- **Performance:** Regular views execute their query every time they are accessed. This can be slow for complex queries on large datasets.
- **Data Warehousing:** Useful for summarizing large amounts of data that don't need to be perfectly real-time.

Definition

A **materialized view** is a database object that contains the results of a query. Unlike a regular view, the data is physically stored, creating a snapshot of the data at a specific point in time.

Creating Materialized Views

Example Scenario

Imagine a Sales table with millions of records. A query to calculate total sales per product category is slow.

Creating a Materialized View (PostgreSQL Syntax)

```
CREATE MATERIALIZED VIEW Sales_Summary AS
SELECT
    ProductCategory,
    SUM(SaleAmount) as TotalSales,
    COUNT(*) as NumberOfSales
FROM Sales
GROUP BY ProductCategory;
```



Maintenance of Materialized Views

The Stale Data Problem

Since a materialized view is a physical snapshot, it does not automatically update when the base tables change. The data becomes "stale".

Refreshing a Materialized View

You must explicitly "refresh" the view to update its data.

```
REFRESH MATERIALIZED VIEW Sales_Summary;
```

Refresh Strategies

- **On-demand:** Run the REFRESH command manually or via a script.
- **Scheduled:** Use a scheduler (like cron or a database job) to refresh the view periodically (e.g., every night).

Indexes in SQL

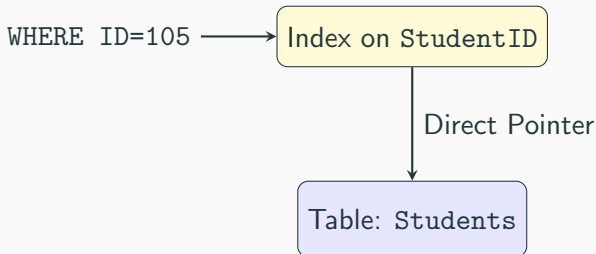
Indices: Motivation and Definition

Motivation

Imagine trying to find a topic in a book with no index. You'd have to scan every page. An index in a database serves the same purpose: it allows the database to find data quickly without scanning the entire table.

Definition

An **index** is a special lookup table that the database search engine can use to speed up data retrieval. It provides a fast pointer to the physical location of a row in a table.



Declaring Indexes: Types and SQL

Standard Index

Creates a simple index on one or more columns.

```
CREATE INDEX idx_student_dept  
ON Students (Department);
```

Unique Index

Ensures that all values in the indexed column(s) are unique. A primary key is automatically a unique index.

```
CREATE UNIQUE INDEX idx_student_email  
ON Students (Email);  
  
-- Fails if any two students have the same email.
```

How to Select Indexes: Tradeoffs

The Core Tradeoff

Indexes speed up data retrieval (SELECT) but slow down data modification (INSERT, UPDATE, DELETE).

Benefits

- Faster SELECT queries with WHERE clauses.
- Faster JOIN operations.
- Can speed up ORDER BY clauses.

Costs

- Slower INSERT, UPDATE, DELETE operations as the index must also be updated.
- Indexes take up disk space.
- Poorly chosen indexes can actually slow down the database.

How to Select Indexes: Tradeoffs

Decision Points

Create indexes on columns that are:

- Frequently used in WHERE clauses.
- Used as foreign keys for JOINS.
- Frequently used in ORDER BY clauses.

Some Useful Index Types with Examples

Single-Column Index

The most common type. Used to speed up queries on a single column.

```
-- Speeds up: SELECT * FROM Students WHERE GPA > 3.8;  
CREATE INDEX idx_gpa ON Students (GPA);
```

Composite (Multi-Column) Index

An index on two or more columns. The order of columns matters.

```
-- Best for queries filtering on Department, then Name  
CREATE INDEX idx_dept_name ON Students (Department, Name);
```

Covering Index

An index that includes all columns required by a query. The database can answer the query using only the index, without ever touching the table.

Thank You